

心电开发板用户手册

(Version 1.0)

蚌医光电

2014-03-15

目 录

1 WDECG 开发板简介.....	3
1.1 WDECG 开发板的特点及优势.....	3
1.2 WDECG 开发板输出功能.....	3
1.3 WDECG 开发板主要技术参数.....	3
1.4 WDECG 开发板的引脚说明.....	3
2 WDECG 开发板硬件连接指南.....	4
3 WDECG 开发板软件开发指南.....	5
3.1 数据类型.....	5
3.1.1 POOR_SIGNAL.....	5
3.1.2 HEART RATE.....	5
3.1.3 RAW Wave Value (16 位)	5
3.1.4 配置字节.....	6
3.1.5 调试输出 1.....	6
3.1.6 调试输出 2.....	6
3.2 WDECG 开发板数据包.....	6
3.2.1 数据包结构.....	6
3.2.2 有效数据格式.....	8
3.2.3 数据包举例.....	9
3.2.4 数据包解析步骤.....	10
3.2.5 一步一步教你解析有效数据.....	10
3.3 解析数据包的例子 (通过 C 语言代码实现)	10
3.4 数据流解析器 C 语言 API.....	12

1 WDECG 开发板简介

WDECG 是一款集心电信号采集、处理和无线传输于一体的微型数字心电开发板，其主要功能是输出心电图 (Electrocardiogram, ECG)、心率 (Heart rate, HR) 和心率变异性 (Heart rate variability, HRV)，可供大学生、研究生、科研工作者及电子爱好者进行科学研究或二次开发使用。

1.1 WDECG 开发板的特点及优势

- 可使用干性电极直接进行心电信号采集
- 可使用像传统医学用的湿性电极，如 Ag/AgCl 一次性心电电极片
- 单通道，2 个测量点
- 蓝牙传输，可方便与手机、平板电脑、笔记本等连接
- 低功耗，适合便携式消费产品
- 最大功耗为 30mA@3.3V
- 原始心电数据采样率 512 Hz

1.2 WDECG 开发板输出功能

- 心电图信号 (0.5-100Hz)
- 心率 (HR)
- 心率变异性 (HRV)

1.3 WDECG 开发板主要技术参数

- 尺寸：3.7cm x 2.4cm
- 采样率：512Hz
- 频率响应：0.5-100Hz
- 输出接口标准：UART (串口) (8 bits, No parity, 1 stop bit)
- 输出波特率：57600
- 静电保护：4kV 接触放电；8kV 隔空放电
- 工作电压：3.5-5.5V

1.4 WDECG 开发板的引脚说明

P1 组 (电源)

Pin 1: VCC

Pin 2: GND

P2 组 (电极)

Pin 1: IN-

Pin 2: GND

Pin 3: IN+

P3 组 (UART/串口)

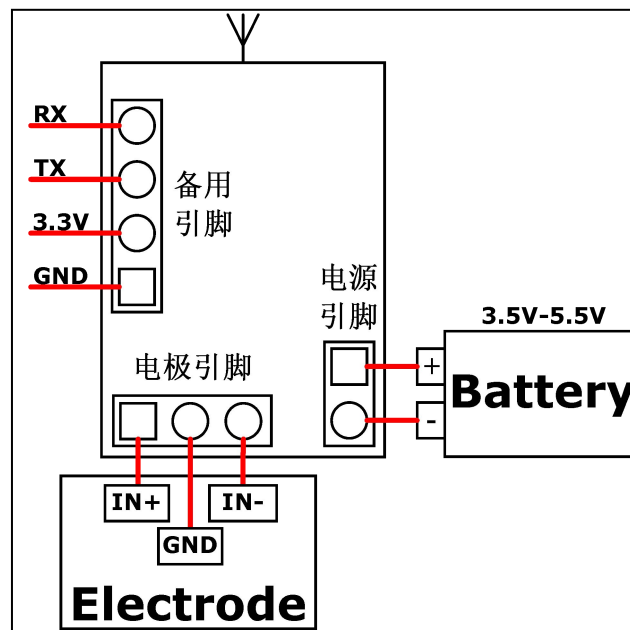
Pin 1: GND

Pin 2: VCCGND

Pin 3: TX

Pin 4: RX

2 WDECG 开发板硬件连接指南



3 WDECG 开发板软件开发指南

3.1 数据类型

3.1.1 POOR_SIGNAL

这个变量是一个字节的无符号整型变量，用来描述 ThinkGear 测得的信号有多差。它的取值范围为 0-200。如果这个变量的值不为 0，就意味着存在着噪声干扰。值越高，意味着噪声越多。特别的指出，值为 200 时，意味着传感器离开了使用者的皮肤。这个变量通常每秒传输一次，表示最新的数据包的信号质量。

信号噪声可能由以下原因导致：

- 1、电极触电没有贴到皮肤上（比如身体没有接触电极）
- 2、传感器接触不良（电极和皮肤之间夹上了头发或者脏东西）
- 3、过度的运动（过度的手指运动挤压了设备）
- 4、过多的环境静电噪声（某些环境中有过多的电信号或人身上静电太多）
- 5、过多的非 ECG 噪声（例如 EMG, EKG, EOG 噪声等）

通常情况下，一定量的噪声是无法避免的，神念科技已经通过算法进行检验和纠正。在某些需要高噪声灵敏度的应用下（比如一些医学研究应用），POOR_SIGNAL 的值可能会需要用到。

默认情况下，此变量默认输出。通常情况下。这个变量每秒输出一次。

3.1.2 HEART RATE

该整型变量用于提供过去一秒钟内心律的值。只有当 POOR_SIGNAL 的值为零时，才输出改变量。因此，当 POOR_SIGNAL 的值为零时，需要忽略此变量的值。

有时候，HEART_RATE 的值会非常高（比如达到 200），或者非常低（比如 20），这可能由多种原因导致。如果按照严重程度排序：

- 1、电极触电没有贴到皮肤上（比如身体没有接触电极）
- 2、传感器接触不良（电极和皮肤之间夹上了头发或者脏东西）
- 3、过度的运动（过度的手指运动挤压了设备）
- 4、过多的环境静电噪声（某些环境中有过多的电信号或人身上静电太多）
- 5、过多的非 ECG 噪声（例如 EMG, EKG, EOG 噪声等）

默认情况下，此变量默认输出。通常情况下。这个变量每秒输出一次。

3.1.3 RAW Wave Value (16 位)

这个变量由两个字节组成，它代表着原始心电图的采样值。该变量的是一个从 -32768-32767 的 16 位整型变量。第一个字节代表着高 8 位，第二个字节代表着低八位。为了还原完整的原始数据，只要把第一个字节左移 8 位，再和第二个字节按位或即可。

```
short raw = (Value[0]<<8) | Value[1];
```

说明一下 Value[0]为高 8 位，Value[1]为低 8 位。

若系统或编程语言中，位运算不方便，下面的算术运算可以实现还原数据的功能：

```
raw = Value[0]*256 + Value[1];
```

```
if ( raw >= 32768) raw = raw -65536;
```

说明一下，raw 是一个在程序中已经被声明的 16 位整型变量，他的取值为-32768-32767。

BMD100 返回的值在-32768-32767 之间。

默认设置下，改变量不输出。如果输出，该变量每秒钟输出 512 次，大约 2ms 一次。

3.1.4 配置字节

改变量由 1 个字节构成。当应用程序读到设置字节以后，应用程序必须把配置字节回传给 BMD100 进行内部设置。如果需要详细信息，请查阅 [App Note - Sending Command Bytes](#) 文档。

3.1.5 调试输出 1

该变量由 5 个字节构成。该变量用于神念科技内部调试。请忽略这个值。

3.1.6 调试输出 2

该变量由 5 个字节构成。该变量用于神念科技内部调试。请忽略这个值。

3.2 WDECG 开发板数据包

TGAM 模块通过一步串行数据流的形式，发送数据。必须对接受到的数据流，进行解析和解释，以便得到前面的章节《[ThinkGear 传输的各类数据](#)》中描述的变量。

一个 ThinkGear 的数据包由三部分构成：

- 1、包头
- 2、实际数据
- 3、校验字节

ThinkGear 包，用于把数据从 TGAM 模块，传输给任意一个接收器（PC、或者微处理器或者其他任何可处理串行数据的移动设备）。由于不同平台、系统、编程语言上，串行 I/O 的 API 都是不一样。所以下文只介绍如何解析串行数据流，使其变成有意义的数。

我们把数据包结构简单化，使其能够灵活使用：通过定义包头和校验字节，使得数据流能够同步、数据能够被完整检查。这样的设计，使得当需要传输跟多种类的数据时，数据可以很简单地添加到数据流中（删除也是一样的道理）。这就意味着，对于任何应用程序，如果升级 TGAM 硬件，可以不修改解析数据流的程序，即使新的硬件会传输新种类的数据。

3.2.1 数据包结构

数据包是由异步串行字节流构成。数据传输可以通过 UART、串行 COM 口、蓝牙、文件

或者其他任何以字节流传输数据的设备来传输。

对于每个包，其开头部分为 包头，中间部分是实际数据，结尾是检验字节，你可以参考下面的格式：

<u>[SYNC][SYNC][PLENGTH]</u>	<u>[PAYLOAD]</u>	<u>[CHKSUM]</u>
数据包头	实际数据	检验变量

[PAYLOAD...]部分的长度不会超过 169 个字节，其中[SYNC]、[PLENGTH]和[CHKSUM]都为 一个字节。这就意味着，一个完整且有效的数据包，至少有 4 个字节的长度（在没有有效数据传回的时候，即是空的时候），至多不超过 173 个字节（当有效数据的长度为 169 个字节）。

正确解析 ThinkGear 数据包的程序，已经给在章节《[一步一步教你解析数据包](#)》中了。

➤ 数据包头

包头由三个字节组成：两个用于识别帧头的字节（0xAA 0xAA），后面跟着一个字节 [PLENGTH]（表示有效数据长度）

<u>[SYNC][SYNC][PLENGTH]</u>
包头

开头的两个[SYNC]字节用于标识一个新数据帧的开始，其值为 0xAA（十进制 170）。要注意到，用于同步的些字节，是两个一样的 0xAA，不是一个，这样能避免当有效数据中包含 170 时，程序错误的识别成了数据包头。虽然在[实际数据](#)中仍然有可能出现两个[SYNC]，但是[PLENGTH]和[CHKSUM]能确保不把包识别错。

字节[PLENGTH]用于说明有效数据的长度，其值为 0-169。任何大于 169 的数，都意味着出错了（PLENGTH 过大）。需要注意到，字节[PLENGTH]是指有效数据的长度，而不是整个数据帧的长度。一个数据帧的长度为[PLENGTH] + 4。

➤ 有效数据

实际数据由许多字节组成。有效数据的长度，等于数据包头中[PLENGTH]的值。有效数据的组成成分，已经在章节《[ThinkGear 数据类型](#)》中阐述过了。有效数据额结构，将会在《有效数据的结构》部分中讲述。要注意到，只有通过校验字节 Checksum 检验合格的数据包。才是有意义的。

➤ 检验字节

字节[CHKSUM]用于检验有效数的正确性。CHKSUM 的定义如下：

- 1、把所有[实际数据](#)的各个字节相加。
- 2、取低 8 位的数。
- 3、把这 8 位数倒置（高低位对换，如把 0000 0001 变成 1000 0000）

接收端接收到的数据包，必须通过前面的 3 个步骤进行检验。如果计算得到的 CHKSUM 于接受到[CHKSUM]不相同，这意味着这一帧的数据有错误，无法使用。如果相等，接收端

将会对有效数据进行分析，得到各类变量。详细的方法将在下面“有效数据格式”部分中给出。

3.2.2 有效数据格式

一旦通过了 [Checksum](#) 的检验，接收端就可以解析实际数据了。实际数据由一系列的数据单元构成（数据单元都是由字符串构成的）。每数据单元都记录着一个变量的信息：变量编号、变量长度、变量实际值。因此，为了解析出实际数据，必须先解析出实际数据中的每一个数据单元，直到实际数据的所有字节都被解析完毕为止。

➤ 数据单元的格式

([EXCODE]...)[CODE] ([VLENGTH]) [VALUE...]
变量编号 变量长度 变量实际值

提示：打括号的字节是有条件地出现的。它们只会出现在某些行数据中，而不是所有的行数据。若需详细信息，请参阅以下说明。

要解析数据单元，先要看数据单元开头的字节[EXCODE]的个数。[EXCODE]的值恒为 0x55。[EXCODE]的个数可能是 0 个，也可能是很多个。

如果数据单元开头一个[EXCODE]也没有的，数据单元就仅由[CODE][VLENGTH][VALUE...]三个字符串构成，其中[CODE]只由一个字节构成，记录着变量的编号（编号与变量的对应关系请查下表），[VLENGTH]只由一个字节构成，记录着变量的长度（当该变量仅有一个字节长度时，数据单元里不会包含该字节），[VALUE...]由一个或多个字节构成（字节数和 VLENGTH 的值相等），用以记录变量的实际值。

为了方便编程，我们基于变量长度、把变量分为两类：单字节变量、多字节变量。

如果 CODE 的值在 0x00 到 0x7F 之间，就意味着该数据单元所传输的变量为单字节变量，也就是说，没有[VLENGTH]字节。字节[CODE]之后，直接就是[VALUE]，且[VALUE]仅有一个字节长度。

如果 CODE 的值比 0x7F 大，就意味着数据单元传输的是多字节变量，也就意味着，在字节[CODE]后，会有字节[VLENGTH]，在[VLENGTH]后才是[VALUE...]，且[VALUE...]的长度和[VLENGTH]的值相同。

如果出现字节[EXCODE]，你需要看一共有几个连续的[EXCODE]，在连续的[EXCODE]之后，第一个不为[EXCODE]的字节即为[VLENGTH]，[VLENGTH]后是[VALUE...]。需要注意到[EXCODE]出现的次数不同，代表的变量不同。还有一点需要注意到，章节《[ThinkGear 数据类型](#)》里提及的变量，都不会用到[EXCODE]。

为了防止日后出现新的 CODE（比如更换了新版的 TGAM 模块），导致解析程序无法正常工作，数据单元被定义成了这种格式。（同理，这样也是为了防止少传数据导致的错误，比如更改 TGAM 模块回传方式）。

正确解析数据包和行数的方法，已经分别在章节《[一步一步教你解析数据包](#)》和《[一步一步教你解数据单元](#)》中给出。

单字节 CODE

[EXCODE]个数	编号[CODE]	长度[VLENGTH]	对应变量
0	0x02	没有字节[VLENGTH]	POOR_SIGNAL (0-200)

0	0x03	没有字节[VLENGTH]	HEART_RATE (0-255)
0	0x08	没有字节[VLENGTH]	CONFIG_BYTE (0-255)

多字节 CODE

[EXCODE]个数	编号[CODE]	长度[VLENGTH]	对应变量
0	0x80	2	POOR_SIGNAL (0-200)
0	0x84	5	HEART_RATE (0-255)
0	0x85	3	CONFIG_BYTE (0-255)
任意个	0x55	-	预留给[EXCODE]
任意个	0xAA	-	预留给[SYNC]

（任何未被列出的 [EXCODE]个数或 CODE，均未被定义、使用，但是可能会在以后添加进表中）。如果需要各变量的详细说明，请参考章节《ThinkGear 数据类型》。

3.2.3 数据包举例

下面是个典型的数据包。除了包头的字节[SYNC]、[PLENGTH]和[CHKSUM]，其他所有的字节（3-34）都属于有效数据。需要注意到，行数据既不会在所有数据包中出现（比如空包），也不会出现在任何特殊的命令下传回的数据包中。如需详细的数据类型信息，请参考章节《ThinkGear 数据类型》。

序号	值	解释
0	0xAA	[SYNC]
1	0xAA	[SYNC]
2	0x12	[PLENGTH]（实际数据长度）18 个字节
3	0x02	[POOR_SIGNAL]
4	0x00	噪声为 0
5	0x03	[HEART_RATE]
6	0xAA	心律为每分钟 170 次
7	0x84	[DEBUG_1]
8	0x05	[VLENGTH] 5 个字节
9	0x00	(1/5) DEGUG_1 起始字节
10	0xF9	(2/5)
11	0x00	(3/5)
12	0x03	(4/5)
13	0x44	(5/5) DEGUG_1 结束字节
14	0x08	[CONFIG_BYTE]
15	0x39	[CONFIG_BYTE]的值
16	0x85	[DEBUG_2]
17	0x03	[VLENGTH] 3 个字节
18	0xFF	(1/5) DEGUG_2 起始字节
19	0xFF	(2/5)
20	0xFF	(3/5) DEGUG_2 结束字节
21	0xC1	[CHKSUM]（其为有效数据求和的低八位的逆）

3.2.4 数据包解析步骤

- 1、不断读取新的字节，知道读到[SYNC]时才执行下一步。
- 2、读取下一个字节的值，确保其为[SYNC]
若不是[SYNC]，返回第一步
否则，执行第三步
- 3、读取下一个字节，将其视作[PLENGTH]
如果[PLENGTH]是 170 ([SYNC])，重复第 3 步
如果[PLENGTH]比 170 大，返回第一步 (PLENGTH 太大)
否则继续第 4 步
- 4、读取[PLENGTH]后面的一个字节 (其属于有效数据)，把其保存到存储空间 (比如数组 `unsigned char payload[256]`)。把其全部加起来 (`checksum += byte`)。
- 5、取其低八位，让后对其进行逆转。下面是 C 语言的代码：

```
checksum &= 0xFF;  
checksum = ~checksum & 0xFF;
```
- 6、读取数据流中最后的字节[CHKSUM]。
如果[CHKSUM]和变量 `checksum` 不相等，回到第一步。
否则，解析有效数据，以获得 ThinkGear 传输的各类变量。
无论怎样，回到第一步。

3.2.5 一步一步教你解析有效数据

重复一下步骤，直到所有[实际数据](#)中的[数据单元](#)解析完毕。

- 1、提取[数据单元](#)开头中，[EXCODE]值的个数。
- 2、提取当前行数据中[CODE]的值。
- 3、如果[CODE] >= 0x80，把下一个字节当成[VLENGTH]来解释。
- 4、基于之前得到的[EXCODE]数量、[CODE]的值、[VLENGTH]的值，解释并处理当前行数据中的字节[VALUE...] (参考 [CODE 定义表](#))。
- 5、如果没有解析完[实际数据](#)，回到第 1 步。另外继续解析下一组[数据单元](#)。

3.3 解析数据包的例子 (通过 C 语言代码实现)

下面是一个示例程序代码，为 C 语言。其数据流读取正确，且数据包不断被传送。为了合适你的程序，请在下边的两部分中，寻找语句 TODO，对其进行修改。

提示：为了简单起见，检查错误函数、标准库函数已被省略。一个正真的应用程序，应当要合适地处理所有的错误。

```
#include <stdio.h>
```

```
#define SYNC 0xAA
```

```
#define EXCODE 0x55
```

```
int parsePayload( unsigned char *payload, unsigned char pLength ) {  
    unsigned char bytesParsed = 0;  
    unsigned char code;
```

```
unsigned char length;
unsigned char extendedCodeLevel;
int i;

/* Loop until all bytes are parsed from the payload[] array... */
while( bytesParsed < pLength ) {

    /* Parse the extendedCodeLevel, code, and length */
    extendedCodeLevel = 0;
    while( payload[bytesParsed] == EXCODE ) {
        extendedCodeLevel++;
        bytesParsed++;
    }
    code = payload[bytesParsed++];
    if( code & 0x80 ) length = payload[bytesParsed++];
    else length = 1;

    /* TODO: Based on the extendedCodeLevel, code, length,
    * and the [CODE] Definitions Table, handle the next
    * "length" bytes of data from the payload as
    * appropriate for your application.
    */
    printf( "EXCODE level: %d CODE: 0x%02X length: %d\n",
        extendedCodeLevel, code, length );
    printf( "Data value(s):" );
    for( i=0; i<length; i++ ) {
        printf( " %02X", payload[bytesParsed+i] & 0xFF );
    }
    printf( "\n" );
    /* Increment the bytesParsed by the length of the Data Value */
    bytesParsed += length;
}
return( 0 );
}

int main( int argc, char **argv ) {
    int checksum;
    unsigned char payload[256];
    unsigned char pLength;
    unsigned char c;
    unsigned char i;

    /* TODO: Initialize 'stream' here to read from a serial data
    * stream, or whatever stream source is appropriate for your
    * application. See documentation for "Serial I/O" for your
    * platform for details.
    */
    FILE *stream = 0;
    stream = fopen( "COM4", "r" );

    /* Loop forever, parsing one Packet per loop... */
    while( 1 ) {

        /* Synchronize on [SYNC] bytes */
        fread( &c, 1, 1, stream );
        if( c != SYNC ) continue;
        fread( &c, 1, 1, stream );
        if( c != SYNC ) continue;
```

```
/* Parse [LENGTH] byte */
while( true ) {
    fread( &pLength, 1, 1, stream );
    if( pLength ^= 170 ) break;
}
if( pLength > 169 ) continue;

/* Collect [PAYLOAD...] bytes */
fread( payload, 1, pLength, stream );

/* Calculate [PAYLOAD...] checksum */
checksum = 0;
for( i=0; i<pLength; i++ ) checksum += payload[i];
checksum &= 0xFF;
checksum = ~checksum & 0xFF;

/* Parse [CKSUM] byte */
fread( &c, 1, 1, stream );

/* Verify [CKSUM] byte against calculated [PAYLOAD...] checksum */
if( c != checksum ) continue;

/* Since [CKSUM] is OK, parse the Data Payload */
parsePayload( payload, pLength );
}
return( 0 );
}
```

3.4 数据流解析器 C 语言 API

ThinkGear 数据解析器 API 是一个函数库，他通过两个简单的函数，实现了抽象出数据的功能。这样，其使程序员不用担心该如何解析数据包和行数据。剩下，需要程序员做的，就是取数据流，放入分析器中，然后定义自己需要的程序变量值（需要从行数据中提取出来的变量）。

ThinkGear 数据流解析器 API 的源代码，是脑电波开发工具（MDT）的一部分。其中包括.h 的头文件，和.c 的源文件。其实在纯 ANSI C 的环境下实现的，能够给所有平台提供最大的可移植性（包括微处理器）。

使用 API 的 3 个步骤：

- 1、定义一个数据处理（回调）函数，处理它们接收到的数据，并解析数据。
- 2、初始化一个 ThinkGear 数据流解析器的结构体，其通过函数 THINKGEAR_initParser() 实现。
- 3、当每个字节，从数据流中读取出来时，用户的程序应当将其传递给函数 THINKGEAR_paresByte()。当每个数据的值解析出来时，此函数将会自动的调用数据处理函数（在第 1 条中定义的）。

下面的部分是从文件 ThinkGearStreamParser.h 节选出来的 API 文档。

常量

```
/*解析类型*/
#define PARSER_TYPE_NULL          0x00
#define PARSER_TYPE_PACKETS      0x00 /*ThinkGear 数据包的数据流字节*/
#define PARSER_TYPE_2BYTERAW     0x00 /*流数据中 2 字节长的原始数据*/
```

```
/*数据 CODE 定义*/
#define PARSER_POOR_SIGNAL_CODE          0x02
#define PARSER_HEARTRATE_CODE            0x03
#define PARSER_RAW_CODE                   0x80
#define PARSER_DEBUGA_CODE                0x08
#define PARSER_DEBUGB_CODE                0x84
#define PARSER_DEBUGC_CODE                0x85
```

THINKGEAR_initParser()

```
/**
 * @程序 parser      通过指针指向一个 ThinkGearStreamParser 结构体对象
 * @程序 parserType  基于 PARSER_TYPE_PACKET 或 PARSER_TYPE_@BYTERAW,
 *                  定义常量 PARSER_TYPE_*
 * @程序 handleDataValueFunc  其为一个用户定义的回调函数。当从数据包中
 *                            解析出数据值时，其会被调用。
 * @程序 customData  其为一个指针，其可以指向任意数据。
 *                  当解析 DataValue 时，被指向的数据
 *                  会被传输到函数 handleDataValueFunc 中。
 * @当@cparser 传入空值时，返回-1。
 * @当@cparserType 无效时，返回-2。
 * @成功时返回 0。
 */
int
THINKGEAR_initParser ( ThinkGearStreamParser *parser, unsigned char parserType,
                      void (*handleDataValueFunc) (
                        unsigned char extendedCodeLevel,
                        unsigned char code, unsigned char numBytes,
                        const unsigned char *value, void *customData),
                      void *customData );
```

THINKGEAR_ParseByte()

```
/**
 * @初始化: 把参数解析器指向 ThinkGearDataParser 对象
 * @从数据流里读取下一个字节
 * @如果 C 解析器为空，返回-1
 * @如果 C 解析器接收到字符串，但是没有通过校验，返回-2
 * @如果 C 解析器解析的包不完整，返回 0
 * @如果 C 解析器成功解析所有变量，返回 1
 *
 */
```

举例

下面是一个 ThinkGearStreamParser API 的示例程序。其与下面的示例程序十分相似。其功能为把收到的数据传输给 stdout:

```
#include <stdio.h>
#include "ThinkGearStreamParser.h"
/**
 * 1) 函数作用于收到的每个行数据。
 */
void
handleDataValueFunc( unsigned char extendeCodeLevel,
                    unsigned char code,
```

```

        unsigned char valueLength,
        const unsigned char *value,
        void *cstomData ) {
if ( extendedCodeLevel == 0 ) {
    switch ( code ) {
        /*[CODE] ATTENTION eSense */
        case ( 0x04 ):
            printf ( "Attention Level: %d\n" , value[0] & 0xFF );
            break;
        /*[CODE] MEDITATION eSense*/
        case ( 0x05 ):
            printf ( "Meditation Level: %d\n" , value[0] & 0xFF );
            break;
        /*Other [CODE]s*/
        default:
            printf ( "EXCODE level %d CODE: 0%&02X vLength: %d\n" ,
                extendedCodeLevel, code, valueLength );
            printf ( "Data value(s):" );
            for ( i=0; i<valueLength; i++ ) printf ( " %02X" , value[i] & 0xFF );
            printf ( "\n" );
        }
    }
}
}
/**
 * 从 COM 口读取 ThinkGear 数据的程序。
 */
int
main ( int argc, char **argv ) {
    /* 2) 初始化 ThinkGear 数据流结构体 */
    ThinkGearStreamParser parser;
    THINKGEAR_initParser ( &parser, PARSER_PACKETS , handleDataValueFunc , NULL );
    /*TODO: 此处初始化“数据流”。
    *这里的数据流可以是 ThinkGear 数据流,
    *或者是任何适合你程序的数据流。
    *详细信息请参考文档“Serial I/O”。
    */
    unsigned char streamByte;
    while ( i ) {
        fread ( &streamByte , 1 , stream );
        THINKGEAR_parseByte ( &parser , streamByte );
    }
}

```

一些需要注意的事:

函数 `handleDataValueFunc()` 应当被快速执行, 这样讲就不会妨碍读取数据流中的数据。一个更有效的 (有用) 的程序, 应当能够把以下线程分开: 读取数据流、调用函数 `handleDataValueFunc()`、定义函数 `handleDataValueFunc()`。而主线程的实际主要任务, 是: 保存显示在屏幕上各变量的值、控制游戏等。线程处理, 不属于本文档阐述范畴。

不同操作系统，从串口中读取数据流的代码不同。通常情况下，如果更换使用平台，学习代码就像是读一篇新的文档一样。串口通信，是本手册外的内容，所以若需详细信息，请查询文档“Serial I/O”。作为替代，你可以使用 ThinkGear Communications Driver (TGCD, ThinkGear 通讯驱动) API。他可以照顾到各个平台，使你能够方便的打开和读取串口。若需要其详细内容，请参考目录为 [developer\tools\2.1\development\guide](#) 的文档 [TGCD API](#)。

为了上面的代码方便理解，省略了许多处理 `error` 的代码。一个正规的程序代码，应当检查所有函数返回的 `error` 代码。请仔细阅读头文件 `ThinkGearStreamPaser.h`，以了解各个函数返回值的详细含义。